

机器学习：机器学习的数学基础

一、概述

我们知道，机器学习的特点就是：以计算机为工具和平台，以数据为研究对象，以学习方法为中心；是概率论、线性代数、数值计算、信息论、最优化理论和计算机科学等多个领域的交叉学科。所以本文就先介绍一下机器学习涉及到的一些最常用的的数学知识

二、线性代数

2-1、标量

一个标量就是一个单独的数，一般用小写的的变量名称表示

2-2、向量

一个向量就是一列数，这些数是有序排列的。通过次序中的索引，我们可以确定每个单独的数。通常会赋予向量粗体的小写名称。当我们需要明确表示向量中的元素时，我们会将元素排

列成一个方括号包围的纵柱：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

我们可以把向量看作空间中的点，每个元素是不同的坐标轴上的坐标。

2-3、矩阵

矩阵是二维数组，其中的每一个元素被两个索引而非一个所确定。我们通常会赋予矩阵粗体的大写变量名称，比如A。如果一个实数矩阵高度为m，宽度为n，那么我们说 $A \in \mathbb{R}^{m \times n}$

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{31} & a_{32} & \cdots & a_{3n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

矩阵这东西在机器学习中就不要太重要了！实际上，如果我们现在有N个用户的数据，每条数据含有M个特征，那其实它对应的就是一个N*M的矩阵呀；再比如，一张图由16*16的像素点组成，那这就是一个16*16的矩阵了。现在才发现，我们大一学的矩阵原理原来这么的有用！要是当时老师讲课的时候先普及一下，也不至于很多同学学矩阵的时候觉得莫名其妙了

2-4、张量

几何代数中定义的张量是基于向量和矩阵的推广，通俗一点理解的话，我们可以将标量视为零阶张量，矢量视为一阶张量，那么矩阵就是二阶张量。

例如，可以将任意一张彩色图片表示成一个三阶张量，三个维度分别是图片的高度、宽度和色彩数据。将这张图用张量表示出来，就是最下方的那张表格：

	0	1	2	3	4	5	6	7	8	9	...	310	311	312	313	314	315	316	317	318	319
0	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
1	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
2	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
3	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]
4	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	...	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]	[1.0, 1.0, 1.0]

其中表的横轴表示图片的宽度值，这里只截取0~319；表的纵轴表示图片的高度值，这里只截取0~4；表格中每个方格代表一个像素点，比如第一行第一列的表格数据为[1.0,1.0,1.0]，代表的就是RGB三原色在图片的这个位置的取值情况（即R=1.0，G=1.0，B=1.0）。

当然我们还可以将这一定义继续扩展，即：我们可以用四阶张量表示一个包含多张图片的数据集，这四个维度分别是：图片在数据集中的编号，图片高度、宽度，以及色彩数据。

张量在深度学习中是一个很重要的概念，因为它是一个深度学习框架中的一个核心组件，后续的所有运算和优化算法几乎都是基于张量进行的。

2-5、范数

有时我们需要衡量一个向量的大小。在机器学习中，我们经常使用被称为范数(norm) 的函数衡量矩阵大小。Lp 范数如下：

$$\|x\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$$

所以：

L1范数：为 $\|x\|$ 向量各个元素绝对值之和

L2范数：为 $\|x\|$ 向量各个元素平方和的开方

这里先说明一下，在机器学习中，L1范数和L2范数很常见，主要用在损失函数中起到一个限制模型参数复杂度的作用，至于为什么要限制模型的复杂度，这又涉及到机器学习中常见的过拟合问题。具体的概念在后续文章中会有详细的说明和推导，大家先记住：这个东西很重要，实际中经常会涉及到，面试中也常会被问到！！

2-6、特征分解

许多数学对象可以通过将它们分解成多个组成部分。特征分解是使用最广的矩阵分解之一，即将矩阵分解成一组特征向量和特征值。

方阵A的特征向量是指与A相乘后相当于对该向量进行缩放的非零向量 ν ：

$$A\nu = \lambda\nu$$

标量被称为这个特征向量对应的特征值。

使用特征分解去分析矩阵A时，得到特征向量构成的矩阵V和特征值构成的向量，我们可以重新将A写作：

$$A = V \text{diag}(\lambda) V^{-1}$$

2-7、奇异值分解（Singular Value Decomposition，SVD）

矩阵的特征分解是有前提条件的，那就是只有对可对角化的矩阵才可以进行特征分解。但实际中很多矩阵往往不满足这一条件，甚至很多矩阵都不是方阵，就是说连矩阵行和列的数目都不相等。这时候怎么办呢？人们将矩阵的特征分解进行推广，得到了一种叫作“矩阵的奇异值分解”的方法，简称SVD。通过奇异分解，我们会得到一些类似于特征分解的信息。

它的具体做法是将一个普通矩阵分解为奇异向量和奇异值。比如将矩阵A分解成三个矩阵的乘积：

$$A = UDV^T$$

假设A是一个mn矩阵，那么U是一个mm矩阵，D是一个mn矩阵，V是一个nn矩阵。

这些矩阵每一个都拥有特殊的结构，其中U和V都是正交矩阵，D是对角矩阵（注意，D不一定是方阵）。对角矩阵D对角线上的元素被称为矩阵A的奇异值。矩阵U的列向量被称为左奇异向量，矩阵V的列向量被称为奇异向量。

SVD最有用的一个性质可能是拓展矩阵求逆到非方矩阵上。另外，SVD可用于推荐系统中

2-8、Moore-Penrose伪逆

对于非方阵而言，其逆矩阵没有定义。假设在下面问题中，我们想通过矩阵A的左逆B来求解线性方程：

$$Ax = y$$

等式两边同时左乘左逆B后，得到：

$$x = By$$

是否存在唯一的映射将A映射到B取决于问题的形式。

如果矩阵A的行数大于列数，那么上述方程可能没有解；如果矩阵A的行数小于列数，那么上述方程可能有多个解。

Moore-Penrose伪逆使我们能够解决这种情况，矩阵A的伪逆定义为：

$$A^+ = \lim_{\alpha \searrow 0} (A^T A + \alpha I)^{-1} A^T$$

但是计算伪逆的实际算法没有基于这个式子，而是使用下面的公式：

$$A^+ = VD^+U^T$$

其中，矩阵U，D和V是矩阵A奇异值分解后得到的矩阵。对角矩阵D的伪逆D+是其非零元素取倒之后再转置得到的。

2-9、几种常用的距离

上面大致说过，在机器学习里，我们的运算一般都是基于向量的，一条用户具有100个特征，那么他对应的就是一个100维的向量，通过计算两个用户对应向量之间的距离值大小，有时候能反映出这两个用户的相似程度。这在后面的KNN算法和K-means算法中很明显。

设有两个n维变量 $A = [x_{11}, x_{12}, \dots, x_{1n}]$ 和 $B = [x_{21}, x_{22}, \dots, x_{2n}]$ ，则一些常用的距离公式定义如下：

1、曼哈顿距离

曼哈顿距离也称为城市街区距离，数学定义如下：

$$d_{12} = \sum_{k=1}^n |x_{1k} - x_{2k}|$$

曼哈顿距离的Python实现：

```
1 from numpy import *
vector1 = mat([1,2,3])
vector2 = mat([4,5,6])
print sum(abs(vector1-vector2))
```

2、欧氏距离

欧氏距离其实就是L2范数，数学定义如下：

$$d_{12} = \sqrt{\sum_{k=1}^n (x_{1k} - x_{2k})^2}$$

实际上，当p=1时，就是曼哈顿距离；当p=2时，就是欧式距离。

欧氏距离的Python实现：

```
1 from numpy import *vector1 = mat([1,2,3])vector2 = mat([4,5,6])print  
sqrt((vector1-vector2)*(vector1-vector2).T)
```

3、闵可夫斯基距离

从严格意义上讲，闵可夫斯基距离不是一种距离，而是一组距离的定义：

$$d_{12} = \sqrt[p]{\sum_{k=1}^n (x_{1k} - x_{2k})^p}$$

实际上，当p=1时，就是曼哈顿距离；当p=2时，就是欧式距离

4、切比雪夫距离

切比雪夫距离就是，即无穷范数，数学表达式如下：

$$d_{12} = \max(|x_{1k} - x_{2k}|)$$

切比雪夫距离的Python实现如下：

```
1 from numpy import *vector1 = mat([1,2,3])vector2 = mat([4,5,6])print  
sqrt(abs(vector1-vector2).max)
```

5、夹角余弦

夹角余弦的取值范围为[-1,1]，可以用来衡量两个向量方向的差异；夹角余弦越大，表示两个向量的夹角越小；当两个向量的方向重合时，夹角余弦取最大值1；当两个向量的方向完全相反时，夹角余弦取最小值-1。

机器学习中用这一概念来衡量样本向量之间的差异，其数学表达式如下：

$$\cos \theta = \frac{AB}{|A||B|} = \frac{\sum_{k=1}^n x_{1k} x_{2k}}{\sqrt{\sum_{k=1}^n x_{1k}^2} \sqrt{\sum_{k=1}^n x_{2k}^2}}$$

夹角余弦的Python实现：

```
1 from numpy import *vector1 = mat([1,2,3])vector2 = mat([4,5,6])print
```

```
dot(vector1,vector2)/(linalg.norm(vector1)*linalg.norm(vector2))
```

6、汉明距离

汉明距离定义的是两个字符串中不相同位数的数目。

例如：字符串‘1111’与‘1001’之间的汉明距离为2。

信息编码中一般应使得编码间的汉明距离尽可能的小。

汉明距离的Python实现：

```
1 from numpy import *matV = mat([1,1,1,1],[1,0,0,1])smstr = nonzero(matV[0]-matV[1])print smstr
```

7、杰卡德相似系数

两个集合A和B的交集元素在A和B的并集中所占的比例称为两个集合的杰卡德相似系数，用符号J(A,B)表示，数学表达式为：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

杰卡德相似系数是衡量两个集合的相似度的一种指标。一般可以将其用在衡量样本的相似度上

8、杰卡德距离

与杰卡德相似系数相反的概念是杰卡德距离，其定义式为：

$$J_{\sigma} = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|}$$

杰卡德距离的Python实现：

```
1 from numpy import *import scipy.spatial.distance as distmatV = mat([1,1,1,1],[1,0,0,1])print dist.pdist(matV,'jaccard')
```

三、概率

3-1、为什么使用概率？

概率论是用于表示不确定性陈述的数学框架，即它是对事物不确定性的度量。

在人工智能领域，我们主要以两种方式来使用概率论。首先，概率法则告诉我们AI系统应该如何推理，所以我们设计一些算法来计算或者近似由概率论导出的表达式。其次，我们可以用概率和统计从理论上分析我们提出的AI系统的行为。

计算机科学的许多分支处理的对象都是完全确定的实体，但机器学习却大量使用概率论。实际上如果你了解机器学习的工作原理你就会觉得这个很正常。因为机器学习大部分时候处理的都是不确定量或随机量。

3-2、随机变量

随机变量可以随机地取不同值的变量。我们通常用小写字母来表示随机变量本身，而用带数字下标的小写字母来表示随机变量能够取到的值。例如， x_1 和 x_2 都是随机变量X可能的取值。

对于向量值变量，我们会将随机变量写成 x ，它的一个值为。就其本身而言，一个随机变量只是对可能的状态的描述；它必须伴随着一个概率分布来指定每个状态的可能性。

随机变量可以是离散的或者连续的。

3-3、概率分布

给定某随机变量的取值范围，概率分布就是导致该随机事件出现的可能性。

从机器学习的角度来看，概率分布就是符合随机变量取值范围的某个对象属于某个类别或服从某种趋势的可能性。

3-4、条件概率

很多情况下，我们感兴趣的是某个事件在给定其它事件发生时出现的概率，这种概率叫条件概率。

我们将给定 $X = x$ 时 $Y = y$ 发生的概率记为 $P(Y = y|X = x)$ ，这个概率可以通过下面的公式来计算：

$$P(Y = y|X = x) = \frac{P(Y = y, X = x)}{P(X = x)}$$

3-5、贝叶斯公式

先看看什么是“先验概率”和“后验概率”，以一个例子来说明：

假设某种病在人群中的发病率是0.001，即1000人中大概会有1个人得病，则有： $P(\text{患病}) = 0.1\%$ ；即：在没有做检验之前，我们预计的患病率为 $P(\text{患病})=0.1\%$ ，这个就叫作"先验概率"。

再假设现在有一种该病的检测方法，其检测的准确率为95%；即：如果真的得了这种病，该检测法有95%的概率会检测出阳性，但也有5%的概率检测出阴性；或者反过来说，但如果没有得病，采用该方法有95%的概率检测出阴性，但也有5%的概率检测为阳性。用概率条件概率表示即为： $P(\text{显示阳性}|\text{患病})=95\%$

现在我们想知道的是：在做完检测显示为阳性后，某人的患病率 $P(\text{患病}|\text{显示阳性})$ ，这个其实就称为"后验概率"。

而这个叫贝叶斯的人其实就是为我们提供了一种可以利用先验概率计算后验概率的方法，我们将其称为“贝叶斯公式”。

这里先了解条件概率公式：

$$P(B|A) = \frac{P(AB)}{P(A)}, P(A|B) = \frac{P(AB)}{P(B)}$$

由条件概率可以得到乘法公式：

$$P(AB) = P(B|A)P(A) = P(A|B)P(B)$$

将条件概率公式和乘法公式结合可以得到：

$$P(B|A) = \frac{P(A|B) \cdot P(B)}{P(A)}$$

再由全概率公式：

$$P(A) = \sum_{i=1}^N P(A|B_i) \cdot P(B_i)$$

代入可以得到贝叶斯公式：

$$P(B_i|A) = \frac{P(A|B_i) \cdot P(B_i)}{\sum_{i=1}^N P(A|B_i) \cdot P(B_i)}$$

在这个例子里就是：

$$\begin{aligned} P(\text{患病}|\text{显示阳性}) &= \frac{P(\text{显示阳性}|\text{患病})P(\text{患病})}{P(\text{显示阳性})} \\ &= \frac{P(\text{显示阳性}|\text{患病})P(\text{患病})}{P(\text{显示阳性}|\text{患病})P(\text{患病}) + P(\text{显示阳性}|\text{无病})P(\text{无病})} \\ &= \frac{95\% \cdot 0.1\%}{95\% \cdot 0.1\% + 5\% \cdot 99.9\%} = 1.86\% \end{aligned}$$

贝叶斯公式贯穿了机器学习中随机问题分析的全过程。从文本分类到概率图模型，其基本分类都是贝叶斯公式

期望、方差、协方差等主要反映数据的统计特征，机器学习的一个很大应用就是数据挖掘等，因此这些基本的统计概念也是很有必要掌握。另外，像后面的EM算法中，就需要用到期望的相关概念和性质。

3-6、期望

在概率论和统计学中，数学期望是试验中每次可能结果的概率乘以其结果的总和。它是最基本的数学特征之一，反映随机变量平均值的大小。

假设X是一个离散随机变量，其可能的取值有： $\{x_1, x_2, \dots, x_n\}$ 各个取值对应的概率取值为： $P(x_k), k = 1, 2, \dots, n$ 则其数学期望被定义为：

$$E(X) = \sum_{k=1}^n x_k P(x_k)$$

假设X是一个连续型随机变量，其概率密度函数为 $P(x)$ 则其数学期望被定义为：

$$E(x) = \int_{-\infty}^{+\infty} x f(x) dx$$

3-7、方差

概率中，方差用来衡量随机变量与其数学期望之间的偏离程度；统计中的方差为样本方差，是各个样本数据分别与其平均数之差的平方和的平均数。数学表达式如下：

$$Var(x) = E\{[x - E(x)]^2\} = E(x^2) - [E(x)]^2$$

3-8、协方差

在概率论和统计学中，协方差被用于衡量两个随机变量X和Y之间的总体误差。数学定义式为：

$$Cov(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

3-9、常见分布函数

1) 0-1分布

0-1分布是单个二值型离散随机变量的分布，其概率分布函数为：

$$P(X = 1) = p, P(X = 0) = 1 - p$$

2) 几何分布

几何分布是离散型概率分布，其定义为：在n次伯努利试验中，试验k次才得到第一次成功的机率。即：前k-1次皆失败，第k次成功的概率。其概率分布函数为：

$$P(X = k) = (1 - p)^{k-1} p$$

性质：

$$E(X) = \frac{1}{p}, Var(X) = \frac{1 - p}{p^2}$$

3) 二项分布

二项分布即重复n次伯努利试验，各次试验之间都相互独立，并且每次试验中只有两种可能的结果，而且这两种结果发生与否相互对立。如果每次试验时，事件发生的概率为p，不发生的概率为1-p，则n次重复独立试验中发生k次的概率为：

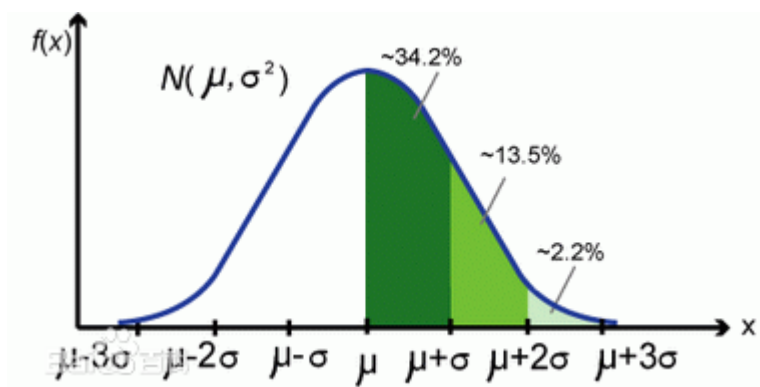
$$P(X = k) = C_n^k p^k (1 - p)^{n-k}$$

性质：

$$E(X) = np, Var(X) = np(1 - p)$$

4) 高斯分布

高斯分布又叫正态分布，其曲线呈钟型，两头低，中间高，左右对称因其曲线呈钟形，如下图所示：



若随机变量 X 服从一个数学期望为 μ ，方差为 σ^2 的正态分布，则我们将其记为： $N(\mu, \sigma^2)$ 。其期望值 μ 决定了正态分布的位置，其标准差 σ （方差的开方）决定了正态分布的幅度

5) 指数分布

指数分布是事件的时间间隔的概率，它的一个重要特征是无记忆性。例如：如果某一元件的寿命的寿命为 T ，已知元件使用了 t 小时，它总共使用至少 $t+s$ 小时的条件概率，与从开始使用时算起它使用至少 s 小时的概率相等。下面这些都属于指数分布：

- 婴儿出生的时间间隔
- 网站访问的时间间隔
- 奶粉销售的时间间隔

指数分布的公式可以从[泊松分布](#)推断出来。如果下一个婴儿要间隔时间 t ，就等同于 t 之内没有任何婴儿出生，即：

$$P(X \geq t) = P(N(t) = 0) = \frac{(\lambda t)^0 \cdot e^{-\lambda t}}{0!} = e^{-\lambda t}$$

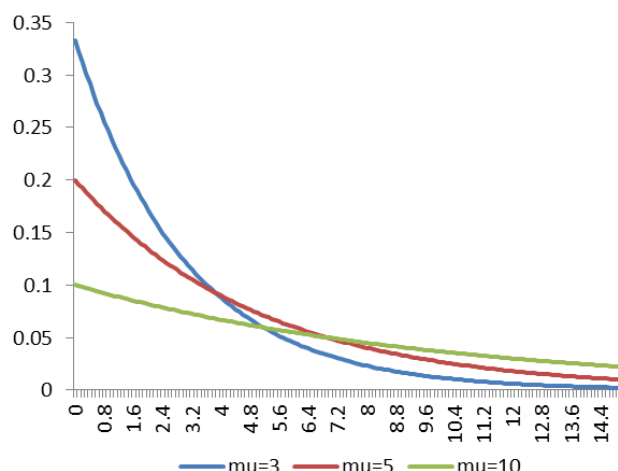
则：

$$P(X \leq t) = 1 - P(X \geq t) = 1 - e^{-\lambda t}$$

如：接下来15分钟，会有婴儿出生的概率为：

$$P(X \leq \frac{1}{4}) = 1 - e^{-3 \cdot \frac{1}{4}} \approx 0.53$$

指数分布的图像如下：



6) 泊松分布

日常生活中，大量事件是有固定频率的，比如：

- 某医院平均每小时出生3个婴儿
- 某网站平均每分钟有2次访问
- 某超市平均每小时销售4包奶粉

它们的特点就是，我们可以预估这些事件的总数，但是没法知道具体的发生时间。已知平均每小时出生3个婴儿，请问下一个小时，会出生几个？有可能一下子出生6个，也有可能一个都不出生，这是我们没法知道的。

泊松分布就是描述某段时间内，事件具体的发生概率。其概率函数为：

$$P(N(t) = n) = \frac{(\lambda t)^n e^{-\lambda t}}{n!}$$

其中：

P表示概率，N表示某种函数关系，t表示时间，n表示数量，1小时内出生3个婴儿的概率，就表示为 $P(N(1) = 3)$ ； λ 表示事件的频率。

还是以上面医院平均每小时出生3个婴儿为例，则；

那么，接下来两个小时，一个婴儿都不出生的概率可以求得为：

$$P(N(2) = 0) = \frac{(3 \cdot 2)^0 \cdot e^{-3 \cdot 2}}{0!} \approx 0.0025$$

同理，我们可以求接下来一个小时，至少出生两个婴儿的概率：

$$P(N(1) \geq 2) = 1 - P(N(1) = 0) - P(N(1) = 1) \approx 0.8$$

【注】上面的指数分布和泊松分布参考了阮一峰大牛的博客：“泊松分布和指数分布：10分钟教程”，在此说明，也对其表示感谢！

3-10、Lagrange乘子法

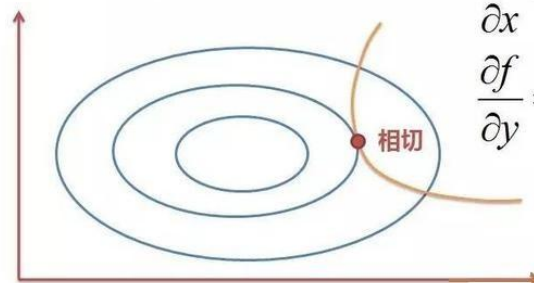
对于一般的求极值问题我们都知道，求导等于0就可以了。但是如果我们不但要求极值，还要求一个满足一定约束条件的极值，那么此时就可以构造Lagrange函数，其实就是把约束项添加到原函数上，然后对构造的新函数求导。

对于一个要求极值的函数 $f(x, y)$ ，图上的蓝圈就是这个函数的等高图，就是说

$f(x, y) = c_1, c_2, \dots, c_n$ 分别代表不同的数值(每个值代表一圈，等高图)，我要找到一组 (x, y) ，使它的 c_i 值越大越好，但是这点必须满足约束条件 $g(x, y)$ （在黄线上）

Lagrange乘子法：

Max $f(x,y)$
约束条件： $g(x,y)=0$



也就是说 $f(x,y)$ 和 $g(x,y)$ 相切，或者说它们的梯度 ∇f 和 ∇g 平行，因此它们的梯度（偏导）成倍数关系；那我么就假设为 λ 倍，然后把约束条件加到原函数后再对它求导，其实就等于满足了下图上的式子。

在支持向量机模型（SVM）的推导中一步很关键的就是利用拉格朗日对偶性将原问题转化为对偶问题

3-11、最大似然估计

最大似然也称为最大概似估计，即：在“模型已定，参数 θ 未知”的情况下，通过观测数据估计未知参数 θ 的一种思想或方法。

其基本思想是：给定样本取值后，该样本最有可能来自参数为何值的总体。即：寻找 $\tilde{\theta}_{ML}$ 使得观测到样本数据的可能性最大

举个例子，假设我们要统计全国人口的身高，首先假设这个身高服从服从正态分布，但是该分布的均值与方差未知。由于没有足够的人力和物力去统计全国每个人的身高，但是可以通过采样（所有的采样要求都是独立同分布的），获取部分人的身高，然后通过最大似然估计来获取上述假设中的正态分布的均值与方差

求极大似然函数估计值的一般步骤：

- 1、写出似然函数

$$L(\theta_1, \theta_2, \dots, \theta_n) = \begin{cases} \prod_{i=1}^n p(x_i; \theta_1, \theta_2, \dots, \theta_n) \\ \prod_{i=1}^n f(x_i; \theta_1, \theta_2, \dots, \theta_n) \end{cases}$$

- 2、对似然函数取对数；
- 3、两边同时求导数；
- 4、令导数为0解出似然方程。

在机器学习中也会经常见到极大似然的影子。比如后面的逻辑斯特回归模型（LR），其核心就是构造对数损失函数后运用极大似然估计

四、信息论

信息论本来是通信中的概念，但是其核心思想“熵”在机器学习中也得到了广泛的应用。比如决策树模型ID3，C4.5中是利用信息增益来划分特征而生成一颗决策树的，而信息增益就是基于这里所说的熵。所以它的重要性也是可想而知

4-1、熵

如果一个随机变量X的可能取值为 $X = \{x_1, x_2, \dots, x_n\}$ ，其概率分布为 $P(X = x_i) = p_i, i = 1, 2, \dots, n$ ，则随机变量X的熵定义为H(X)：

$$H(X) = - \sum_{i=1}^n P(x_i) \log P(x_i) = \sum_{i=1}^n P(x_i) \frac{1}{\log P(x_i)}$$

4-2、联合熵

两个随机变量X和Y的联合分布可以形成联合熵，定义为联合自信息的数学期望，它是二维随机变量XY的不确定性的度量，用H(X,Y)表示：

$$H(X, Y) = - \sum_{i=1}^n \sum_{j=1}^n P(x_i, y_j) \log P(x_i, y_j)$$

4-3、条件熵

在随机变量X发生的前提下，随机变量Y发生新带来的熵，定义为Y的条件熵，用H(Y|X)表示：

$$H(Y|X) = - \sum_{x,y} P(x, y) \log P(y|x)$$

条件熵用来衡量在已知随机变量X的条件下，随机变量Y的不确定性。

实际上，熵、联合熵和条件熵之间存在以下关系：

$$H(Y|X) = H(X, Y) - H(X)$$

推导过程如下：

$$\begin{aligned}
& H(X, Y) - H(X) \\
&= -\sum_{x,y} p(x, y) \log p(x, y) + \sum_x p(x) \log p(x) \\
&= -\sum_{x,y} p(x, y) \log p(x, y) + \sum_x \left(\sum_y p(x, y) \right) \log p(x) \\
&= -\sum_{x,y} p(x, y) \log p(x, y) + \sum_{x,y} p(x, y) \log p(x) \\
&= -\sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)} \\
&= -\sum_{x,y} p(x, y) \log p(y | x)
\end{aligned}$$

其中：

- 第二行推到第三行的依据是边缘分布 $P(x)$ 等于联合分布 $P(x,y)$ 的和
- 第三行推到第四行的依据是把公因子 $\log P(x)$ 乘进去，然后把 x,y 写在一起
- 第四行推到第五行的依据是：因为两个sigma都有 $P(x,y)$ ，故提取公因子 $P(x,y)$ 放到外边，然后把里边的 $-(\log P(x,y) - \log P(x))$ 写成 $-\log(P(x,y) / P(x))$
- 第五行推到第六行的依据是： $P(x,y) = P(x) * P(y|x)$ ，故 $P(x,y) / P(x) = P(y|x)$

4-4、相对熵

相对熵又称互熵、交叉熵、KL散度、信息增益，是描述两个概率分布 P 和 Q 差异的一种方法，记为 $D(P||Q)$ 。在信息论中， $D(P||Q)$ 表示当用概率分布 Q 来拟合真实分布 P 时，产生的信息损耗，其中 P 表示真实分布， Q 表示 P 的拟合分布。

对于一个离散随机变量的两个概率分布 P 和 Q 来说，它们的相对熵定义为：

$$D(P||Q) = \sum_{i=1}^n P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$

注意： $D(P||Q) \neq D(Q||P)$

相对熵又称KL散度(Kullback-Leibler divergence)，KL散度也是一个机器学习中常考的概念

4-5、互信息

两个随机变量 X ， Y 的互信息定义为 X ， Y 的联合分布和各自独立分布乘积的相对熵称为互信息，用 $I(X,Y)$ 表示。互信息是信息论里一种有用的信息度量方式，它可以看成是一个随机变量中包含的关于另一个随机变量的信息量，或者说是一个随机变量由于已知另一个随机变量而减少的不肯定性

$$I(X, Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \frac{P(x, y)}{P(x)P(y)}$$

互信息、熵和条件熵之间存在以下关系： $H(Y|X) = H(Y) - I(X, Y)$

推导过程如下：

$$\begin{aligned} & H(Y) - I(X, Y) \\ &= -\sum_y p(y) \log p(y) - \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \\ &= -\sum_y \left(\sum_x p(x, y) \right) \log p(y) - \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \\ &= -\sum_{x,y} p(x, y) \log p(y) - \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \\ &= -\sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)} \\ &= -\sum_{x,y} p(x, y) \log p(y|x) \\ &= H(Y|X) \end{aligned}$$

通过上面的计算过程发现有： $H(Y|X) = H(Y) - I(X, Y)$ ，又由前面条件熵的定义有： $H(Y|X) = H(X, Y) - H(X)$ ，于是有 $I(X, Y) = H(X) + H(Y) - H(X, Y)$ ，此结论被多数文献作为互信息的定义

4-6、最大熵模型

最大熵原理是概率模型学习的一个准则，它认为：学习概率模型时，在所有可能的概率分布中，熵最大的模型是最好的模型。通常用约束条件来确定模型的集合，所以，最大熵模型原理也可以表述为：在满足约束条件的模型集合中选取熵最大的模型。

前面我们知道，若随机变量 X 的概率分布是 $P(x_i)$ ，则其熵定义如下：

$$H(X) = -\sum_{i=1}^n P(x_i) \log P(x_i) = \sum_{i=1}^n P(x_i) \frac{1}{\log P(x_i)}$$

熵满足下列不等式：

$$0 \leq H(X) \leq \log |X|$$

式中， $|X|$ 是 X 的取值个数，当且仅当 X 的分布是均匀分布时右边的等号成立。也就是说，当 X 服从均匀分布时，熵最大

直观地看，最大熵原理认为：要选择概率模型，首先必须满足已有的事实，即约束条件；在没有更多信息的情况下，那些不确定的部分都是“等可能的”。最大熵原理通过熵的最大化来表示等可能性；“等可能”不易操作，而熵则是一个可优化的指标

五、数值计算

5-1、上溢和下溢

在数字计算机上实现连续数学的基本困难是：我们需要通过有限数量的位模式来表示无限多的实数，这意味着我们在计算机中表示实数时几乎都会引入一些近似误差。在许多情况下，这仅仅是舍入误差。如果在理论上可行的算法没有被设计为最小化舍入误差的累积，可能会在实践中失效，因此舍入误差是有问题的，特别是在某些操作复合时

一种特别毁灭性的舍入误差是下溢。当接近零的数被四舍五入为零时发生下溢。许多函数会在其参数为零而不是一个很小的正数时才会表现出质的不同。例如，我们通常要避免被零除

另一个极具破坏力的数值错误形式是上溢(overflow)。当大量级的数被近似为或时发生上溢。进一步的运算通常将这些无限值变为非数字

必须对上溢和下溢进行数值稳定的一个例子是softmax 函数。softmax 函数经常用于预测与multinoulli分布相关联的概率，定义为：

$$P(y_i = k|x_i) = \frac{e^{w_k x_i + b}}{1 + \sum_{k=1}^{K-1} e^{w_k x_i + b}}, k = 1, 2, \dots, K - 1$$

softmax 函数在多分类问题中非常常见。这个函数的作用就是使得在负无穷到0的区间趋向于0，在0到正无穷的区间趋向于1。上面表达式其实是多分类问题中计算某个样本 x_i 的类别标签 y_i 属于K个类别的概率，最后判别 y_i 所属类别时就是将其归为对应概率最大的那一个。

当式中 $w_k x_i + b$ 的都是很小的负数时， $e^{w_k x_i + b}$ 就会发生下溢，这意味着上面函数的分母会变成0，导致结果是未定的；同理，当式中的 $w_k x_i + b$ 是很大的正数时， $e^{w_k x_i + b}$ 就会发生上溢导致结果是未定的

5-2、计算复杂性与NP问题

1、算法复杂性

现实中大多数问题都是离散的数据集，为了反映统计规律，有时数据量很大，而且多数目标函数都不能简单地求得解析解。这就带来一个问题：算法的复杂性。

算法理论被认为是解决各类现实问题的方法论。衡量算法有两个重要的指标：时间复杂度和空间复杂度，这是对算法执行所需要的两类资源——时间和空间的估算。

一般，衡量问题是否可解的重要指标是：该问题能否在多项式时间内求解，还是只能在指数时间内求解？在各类算法理论中，通常使用多项式时间算法即可解决的问题看作是易解问题，需要指数时间算法解决的问题看作是难解问题。

指数时间算法的计算时间随着问题规模的增长而呈指数化上升，这类问题虽然有解，但并不适用于大规模问题。所以当前算法研究的一个重要任务就是将指数时间算法变换为多项式时间算法。

2、确定性和非确定性

除了问题规模与运算时间的比较，衡量一个算法还需要考虑确定性和非确定性的概念。

这里先介绍一下“自动机”的概念。自动机实际上是指一种基于状态变化进行迭代的算法。在算法领域常把这类算法看作一个机器，比较知名的有图灵机、玻尔兹曼机、支持向量机等。

所谓确定性，是指针对各种自动机模型，根据当时的状态和输入，若自动机的状态转移是唯一确定的，则称确定性；若在某一时刻自动机有多个状态可供选择，并尝试执行每个可选择的状态，则称为非确定性。

换个说法就是：确定性是程序每次运行时产生下一步的结果是唯一的，因此返回的结果也是唯一的；非确定性是程序在每个运行时执行的路径是并行且随机的，所有路径都可能返回结果，也可能只有部分返回结果，也可能不返回结果，但是只要有一个路径返回结果，那么算法就结束。

在求解优化问题时，非确定性算法可能会陷入局部最优。

3、NP问题

有了时间上的衡量标准和状态转移的确定性与非确定性的概念，我们来定义一下问题的计算复杂度。

P类问题就是能够以多项式时间的确定性算法来对问题进行判定或求解，实现它的算法在每个运行状态都是唯一的，最终一定能够确定一个唯一的结果——最优的结果。

NP问题是指可以用多项式时间的非确定性算法来判定或求解，即这类问题求解的算法大多是非确定性的，但时间复杂度有可能是多项式级别的。

但是，NP问题还有一个子类称为NP完全问题，它是NP问题中最难的问题，其中任何一个问题至今都没有找到多项式时间的算法。

机器学习中多数算法都是针对NP问题（包括NP完全问题）的。

5-3、数值计算

上面已经分析了，大部分实际情况中，计算机其实都只能做一些近似的数值计算，而不可能找到一个完全精确的值，这其实有一门专门的学科来研究这个问题，这门学科就是——数值分析（有时也叫作“计算方法”）；运用数值分析解决问题的过程为：实际问题→数学模型→数值计算方法→程序设计→上机计算求出结果。

计算机在做这些数值计算的过程中，经常会涉及到的一个东西就是“迭代运算”，即通过不停的迭代计算，逐渐逼近真实值（当然是要在误差收敛的情况下）。

六、最优化

本节介绍机器学习中的一种重要理论——最优化方法

6-1、最优化理论

无论做什么事，人们总希望以最小的代价取得最大的收益。在解决一些工程问题时，人们常会遇到多种因素交织在一起与决策目标相互影响的情况；这就促使人们创造一种新的数学理论来应对这一挑战，也因此，最早的优化方法——线性规划诞生了。

在李航博士的《统计学习方法》中，其将机器学习总结为如下表达式：

机器学习 = 模型 + 策略 + 算法

可以看得出，算法在机器学习中的重要性。实际上，这里的算法指的就是优化算法。在面试机器学习的岗位时，优化算法也是一个特别高频的问题，大家如果真的想学好机器学习，那还是需要重视起来的。

6-2、最优化问题的数学描述

最优化的基本数学模型如下：

$$\begin{aligned} \min \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & h_i(\mathbf{x}) = 0 \\ & g_j(\mathbf{x}) \leq 0 \end{aligned}$$

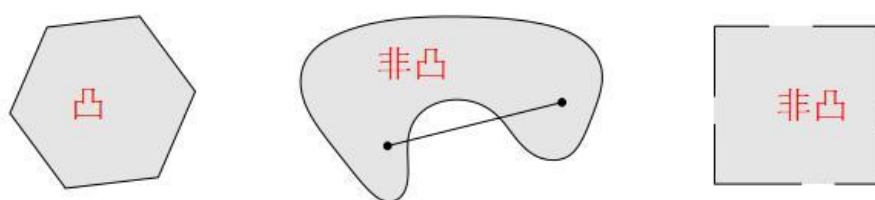
它有三个基本要素，即：

- 设计变量： x 是一个实数域范围内的 n 维向量，被称为决策变量或问题的解；
- 目标函数： $f(x)$ 为目标函数；
- 约束条件： $h_i(x) = 0$ 称为等式约束， $g_i(x) \leq 0$ 为不等式约束， $i = 0, 1, 2, \dots$

6-3、凸集与凸集分离定理

1、凸集

实数域 R 上（或复数 C 上）的向量空间中，如果集合 S 中任两点的连线上的点都在 S 内，则称集合 S 为凸集，如下图所示：



数学定义为：

设集合 $D \subset R^n$ ，若对于任意两点 $x, y \in D$ ，及实数 $\lambda(0 \leq \lambda \leq 1)$ 都有：

$$\lambda x + (1 - \lambda)y \in D$$

则称集合 D 为凸集

2、超平面和半空间

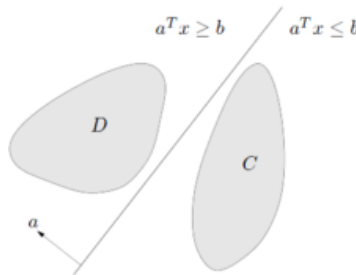
实际上，二维空间的超平面就是一条线（可以是曲线），三维空间的超平面就是一个面（可以是曲面）。其数学表达式如下：

超平面： $H = \{x \in R^n | a_1 + a_2 + \dots + a_n = b\}$

半空间： $H^+ = \{x \in R^n | a_1 + a_2 + \cdots + a_n \geq b\}$

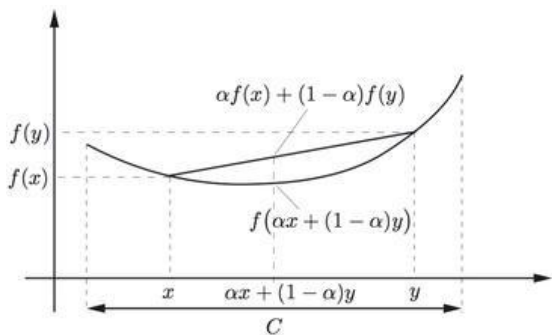
3、凸集分离定理

所谓两个凸集分离，直观地看是指两个凸集合没有交叉和重合的部分，因此可以用一张超平面将两者隔在两边，如下图所示：



4、凸函数

凸函数就是一个定义域在某个向量空间的凸子集C上的实值函数。



数学定义为：

对于函数 $f(x)$ ，如果其定义域 C 是凸的，且对于 $\forall x,y \in C, 0 \leq \alpha \leq 1$,

有：

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

则 $f(x)$ 是凸函数

注：如果一个函数是凸函数，则其局部最优点就是它的全局最优点。这个性质在机器学习算法优化中有很重要的应用，因为机器学习模型最后就是在求某个函数的全局最优点，一旦证明该函数（机器学习里面叫“损失函数”）是凸函数，那相当于我们只用求它的局部最优点了

6-4、梯度下降算法

1、引入

前面讲数值计算的时候提到过，计算机在运用迭代法做数值计算（比如求解某个方程组的解）时，只要误差能够收敛，计算机最后经过一定次数的迭代后是可以给出一个跟真实解很接近的结果的。

这里进一步提出一个问题，如果我们得到的目标函数是非线性的情况下，按照哪个方向迭代求解误差的收敛速度会最快呢？

答案就是沿梯度方向。这就引入了我们的梯度下降法

2、梯度下降法

在多元微分学中，梯度就是函数的导数方向。

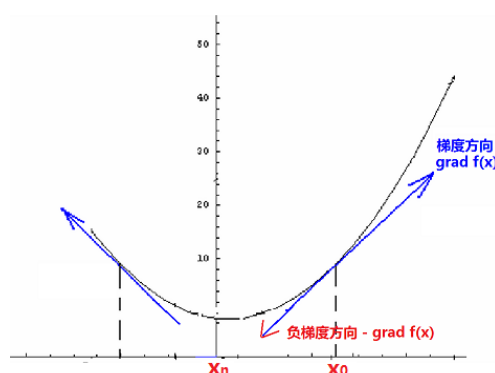
梯度法是求解无约束多元函数极值最早的数值方法，很多机器学习的常用算法都是以它作为算法框架，进行改进而导出更为复杂的优化方法。

在求解目标函数 $f(x)$ 的最小值时，为求得目标函数的一个凸函数，在最优化方法中被表示为：

$$\min f(x)$$

根据导数的定义，函数 $f(x)$ 的导函数就是目标函数在上的变化率。在多元的情况下，目标函数

$f(x, y, z)$ 在某点的梯度 $\text{grad}f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$ 是一个由各个分量的偏导数构成的向量，负梯度方向是 $f(x, y, z)$ 减小最快的方向



如上图所示，当 $f(x)$ 需要求的最小值时（机器学习中的 $f(x)$ 一般就是损失函数，而我们的目标就是希望损失函数最小化），我们就可以先任意选取一个函数的初始点 x_0 （三维情况就是 (x_0, y_0, z_0) ），让其沿着途中红色箭头（负梯度方向）走，依次到 $x_1, x_2, \dots, x_n$ （迭代n次）这样可最快达到极小值点

梯度下降法过程如下：

输入：目标函数 $f(x)$ ，梯度函数 $g(x) = \text{grad}f(x)$ ，计算精度 ε

输出： $f(x)$ 的极小值点 x^*

- 1、任取初始值 x_0 ，置 $k = 0$ ；
- 2、计算 $f(x_k)$ ；
- 3、计算梯度 $g_k = \text{grad}f(x_k)$ ，当 $\|g_k\| < \varepsilon$ 时停止迭代，令 $x^* = x_k$ ；
- 4、否则令 $P_k = -g_k$ ，求 λ_k 使 $f(x_{k+1}) = \min f(x_k + \lambda_k P_k)$ ；
- 5、置 $x_{k+1} = x_k + \lambda_k P_k$ ，计算 $f(x_{k+1})$ ，当 $\|f(x_{k+1}) - f(x_k)\| < \varepsilon$ 或 $\|x_{k+1} - x_k\| < \varepsilon$ 时，停止迭代，令 $x^* = x_{k+1}$ ；
- 6、否则，置 $k = k + 1$ ，转3。

6-5、随机梯度下降算法

上面可以看到，在梯度下降法的迭代中，除了梯度值本身的影响外，还有每一次取的步长也很关键：步长值取得越大，收敛速度就会越快，但是带来的可能后果就是容易越过函数的最优点，导致发散；步长取太小，算法的收敛速度又会明显降低。因此我们希望找到一种比较好的方法能够平衡步长。

随机梯度下降法并没有新的算法理论，仅仅是引进了随机样本抽取方式，并提供了一种动态步长取值策略。目的就是既要优化精度，又要满足收敛速度。

也就是说，上面的批量梯度下降法每次迭代时都会计算训练集中所有的数据，而随机梯度下降法每次迭代只是随机取了训练集中的一部分样本数据进行梯度计算，这样做最大的好处是可以避免有时候陷入局部极小值的情况（因为批量梯度下降法每次都使用全部数据，一旦到了某个局部极小值点可能就停止更新了；而随机梯度法由于每次都是随机取部分数据，所以就算局部极小值点，在下一步也还是可以跳出）

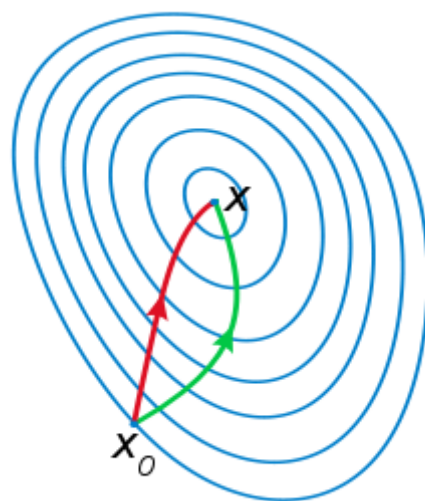
两者的关系可以这样理解：随机梯度下降方法以损失很小的一部分精确度和增加一定数量的迭代次数为代价，换取了总体的优化效率的提升。增加的迭代次数远远小于样本的数量

6-6、牛顿法

1、牛顿法介绍

牛顿法也是求解无约束最优化问题常用的方法，最大的优点是收敛速度快。

从本质上去看，牛顿法是二阶收敛，梯度下降是一阶收敛，所以牛顿法就更快。通俗地说，比如你想找一条最短的路径走到一个盆地的最底部，梯度下降法每次只从你当前所处位置选一个坡度最大的方向走一步，牛顿法在选择方向时，不仅会考虑坡度是否够大，还会考虑你走了一步之后，坡度是否会变得更大。所以，可以说牛顿法比梯度下降法看得更远一点，能更快地走到最底部。



或者从几何上说，牛顿法就是用一个二次曲面去拟合你当前所处位置的局部曲面，而梯度下降法是用一个平面去拟合当前的局部曲面，通常情况下，二次曲面的拟合会比平面更好，所以牛顿法选择的下降路径会更符合真实的最优下降路径

2、牛顿法的推导

将目标函数 $f(x)$ 在 x_k 处进行二阶泰勒展开，可得：

$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2}f''(x_k)(x - x_k)^2$$

因为目标函数 $f(x)$ 有极值的必要条件是在极值点处一阶导数为0，即： $f'(x) = 0$

所以对上面的展开式两边同时求导（注意 x 才是变量， x_k 是常量 $f'(x_k), f''(x_k)$ 都是常量），并令 $f'(x) = 0$ 可得：

$$f'(x_k) + f''(x_k)(x - x_k) = 0$$

即：

$$x = x_k - \frac{f'(x_k)}{f''(x_k)}$$

于是可以构造如下的迭代公式：

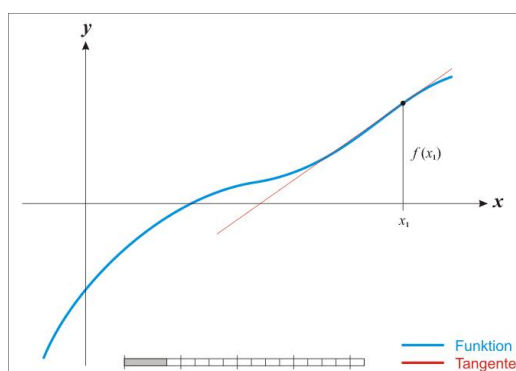
$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

于是可以构造如下的迭代公式：

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

这样，我们就可以利用该迭代式依次产生的序列 $\{x_1, x_2, \dots, x_k\}$ 才逐渐逼近 $f(x)$ 的极小值点了

牛顿法的迭代示意图如下：



上面讨论的是2维情况，高维情况的牛顿迭代公式是：

$$\mathbf{x}_{n+1} = \mathbf{x}_n - [Hf(\mathbf{x}_n)]^{-1} \nabla f(\mathbf{x}_n), n \geq 0.$$

式中， ∇f 是 $f(x)$ 的梯度，即：

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{bmatrix}$$

H是Hessen矩阵，即：

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

3、牛顿法的过程

- 1、给定初值 x_0 和精度阈值 ε ，并令 $k = 0$ ；
- 2、计算 x_k 和 H_k ；
- 3、若 $\|g_k\| < \varepsilon$ 则停止迭代；否则确定搜索方向： $d_k = -H_k^{-1} \cdot g_k$ ；
- 4、计算新的迭代点： $x_{k+1} = x_k + d_k$ ；
- 5、令 $k = k + 1$ ，转至2。

6-7、阻尼牛顿法

1、引入

注意到，牛顿法的迭代公式中没有步长因子，是定步长迭代。对于非二次型目标函数，有时候会出现 $f(x_{k+1}) > f(x_k)$ 的情况，这表明，原始牛顿法不能保证函数值稳定的下降。在严重的情况下甚至会造成序列发散而导致计算失败。

为消除这一弊病，人们又提出阻尼牛顿法。阻尼牛顿法每次迭代的方向仍然是 x_k ，但每次迭代会沿此方向做一维搜索，寻求最优的步长因子 λ_k ，即：

$$\lambda_k = \min f(x_k + \lambda d_k)$$

2、算法过程

- 1、给定初值 x_0 和精度阈值 ε ，并令 $k = 0$ ；
- 2、计算 g_k （ $f(x)$ 在 x_k 处的梯度值）和 H_k ；
- 3、若 $\|g_k\| < \varepsilon$ 则停止迭代；否则确定搜索方向： $d_k = -H_k^{-1} \cdot g_k$ ；
- 4、利用 $d_k = -H_k^{-1} \cdot g_k$ 得到步长 λ_k ，并令 $x_{k+1} = x_k + \lambda_k d_k$ ；
- 5、令 $k = k + 1$ ，转至2。

6-8、拟牛顿法

1、概述

由于牛顿法每一步都要求解目标函数的Hessen矩阵的逆矩阵，计算量比较大（求矩阵的逆运算量比较大），因此提出一种改进方法，即通过正定矩阵近似代替Hessen矩阵的逆矩阵，简化这一计算过程，改进后的方法称为拟牛顿法

2、拟牛顿法的推导

先将目标函数 x_{k+1} 在处展开，得到

$$f(x) = f(x_{k+1}) + f'(x_{k+1})(x - x_{k+1}) + \frac{1}{2}f''(x_{k+1})(x - x_{k+1})^2$$

两边同时取梯度，得：

$$f'(x) = f'(x_{k+1}) + f''(x_{k+1})(x - x_{k+1})$$

取上式中的 $x = x_k$ ，得：

$$f'(x_k) = f'(x_{k+1}) + f''(x_{k+1})(x - x_{k+1})$$

即：

$$g_{k+1} - g_k = H_{k+1} \cdot (x_{k+1} - x_k)$$

可得：

$$H_k^{-1} \cdot (g_{k+1} - g_k) = x_{k+1} - x_k$$

上面这个式子称为“拟牛顿条件”，由它来对Hessen矩阵做约束